CS202 – Observer pattern

APCS 2025
Nguyễn Đình Minh Huy - 25125083
Trần Gia Huy - 25125084

Agenda

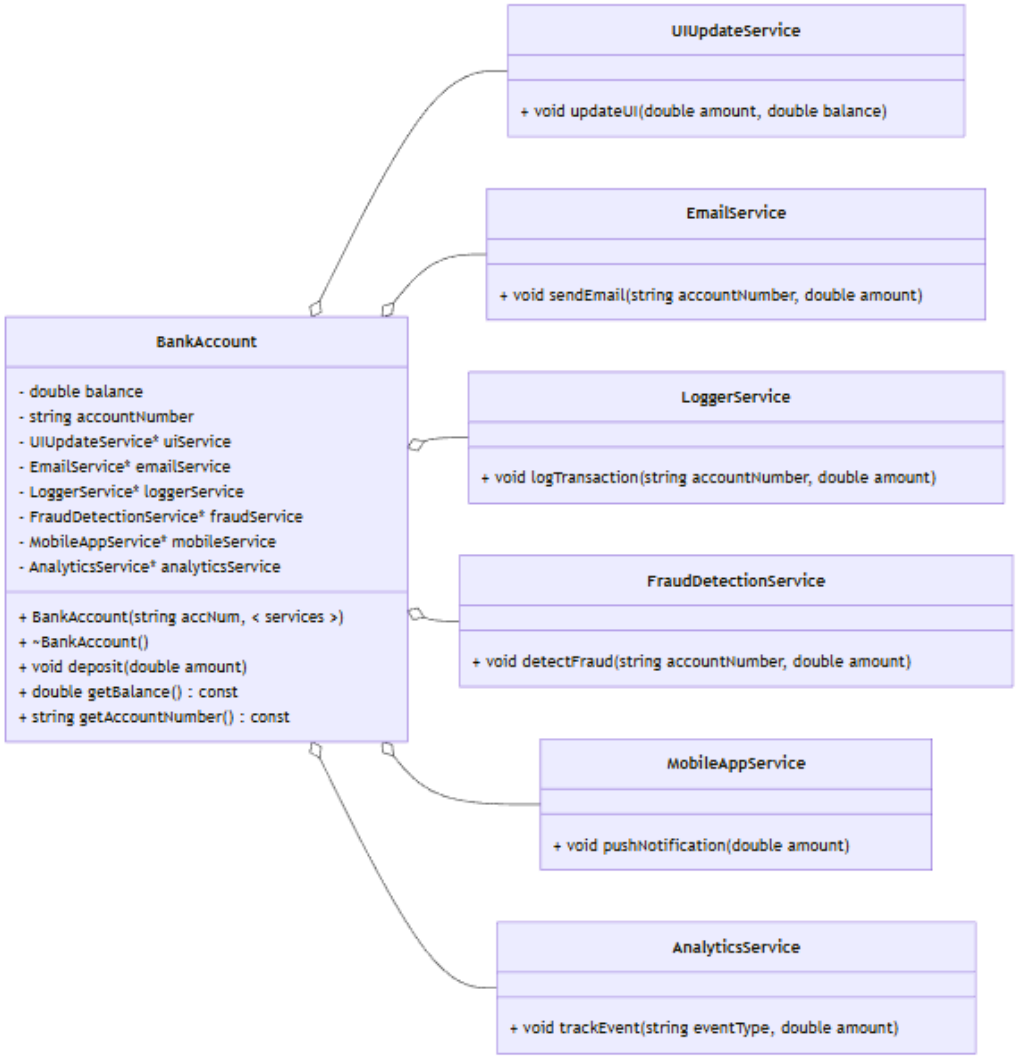
1. Real-world problem
2. Naïve solution & Drawback
3. Observer pattern
4. Class diagram
5. Code implementation & Demo
6. Usage with Interpreter and Mediator design pattern
7. Pros & Cons
8. Quiz
9. Conclusion

Real-world problem

We are building a bank management system. We need to store users' accounts and let them deposit their money into their accounts. However, when we call the deposit, aside from adding money, we are required to do other procedures such as logging, notifying, emailing,… These are from other services that we don't have full control.

Naïve solution

When BankAccount::deposit is called, it invokes all the required services, which are saved as pointers.



Naïve solution - Drawback

However, there are some problems with this approach:

- Tight coupling: The BankAccount must know the exact concrete types of services and their specific update methods.
- Violates the Open-Closed Principle (OCP): If we want to change or add a service, we must modify the code in BankAccount.
- Rigid and hard to scale: Changing and modifying either the service or the BankAccount are much more complex.

We notice that the services have similar parameters, but not exactly the same. If we have a general structure, we won't need to know what each of them need.

Another thing is that the BankAccount don't care what we need to call as they don't need the result from invoking the services. => We only need a list of services to call when depositing.

=> Observer pattern

Observer Design Pattern - Intuition

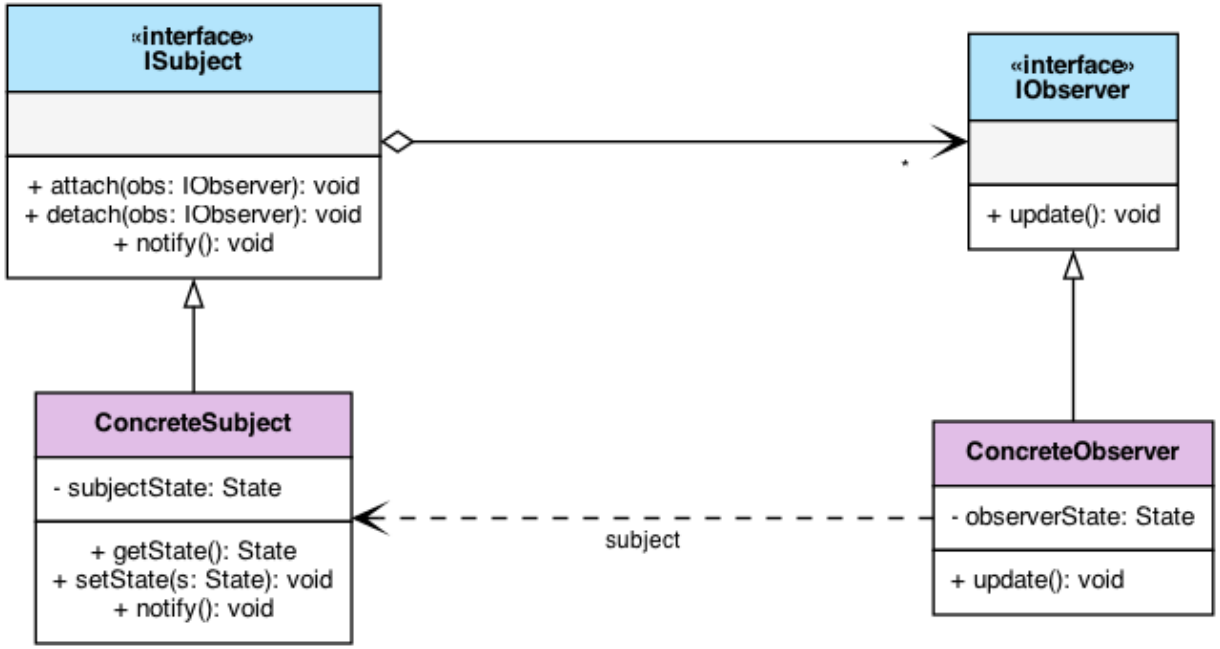
Imagine a popular Youtube channel or a digital newsletter.

- Subscribers should not have to constantly ask, "Is there a new video?" or "Is there a new edition?".
- The Publisher maintains a list of Subscribers. When new content is published, the Publisher automatically pushes a notification to everyone on that list.
- The pattern shifts the responsibility of tracking state changes from the data consumers to the data source, ensuring instant updates without wasteful continuous checks.

In Observer Design Pattern:

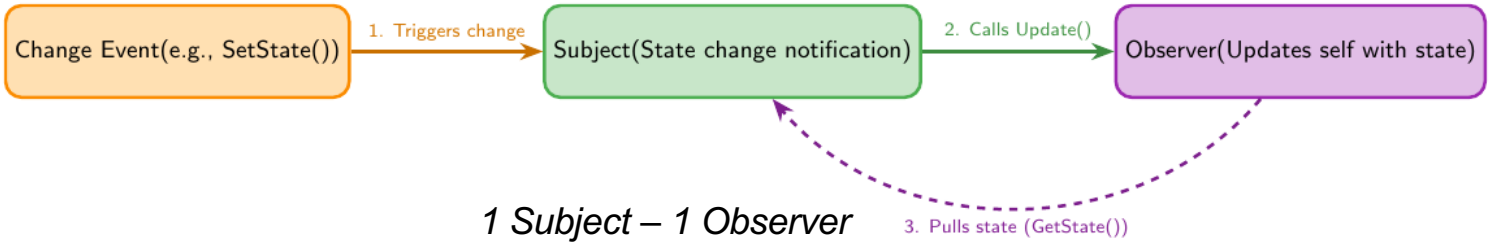
Subject = Publisher ; Observer = Subscriber

General Class Diagram

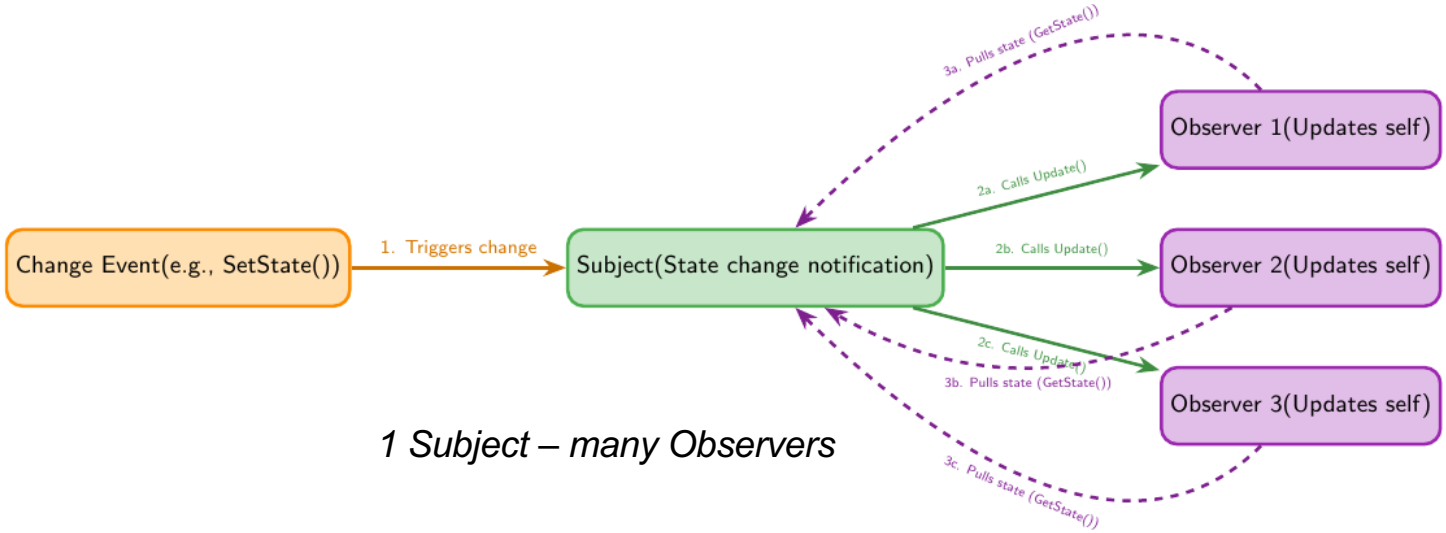


ISubject: Maintains a list of observers. Implements *attach()*, *detach()*, and broadcasts *notify()* when its state changes.

IObserver: Exposes an *update()* method that the subject calls during a notification.

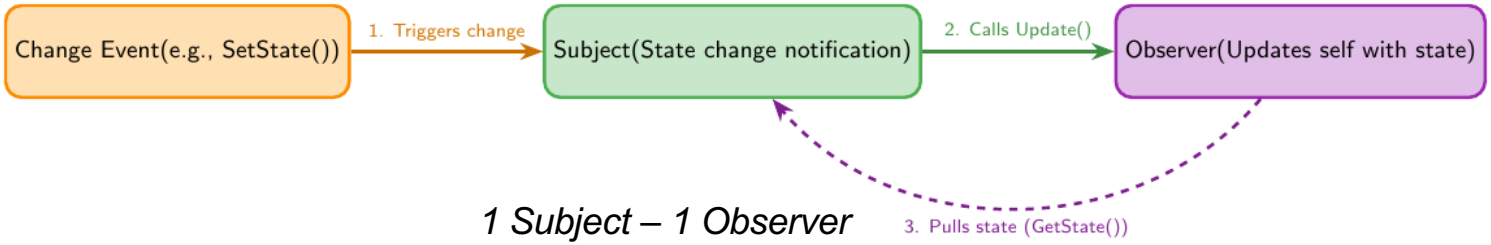


1 Subject – 1 Observer

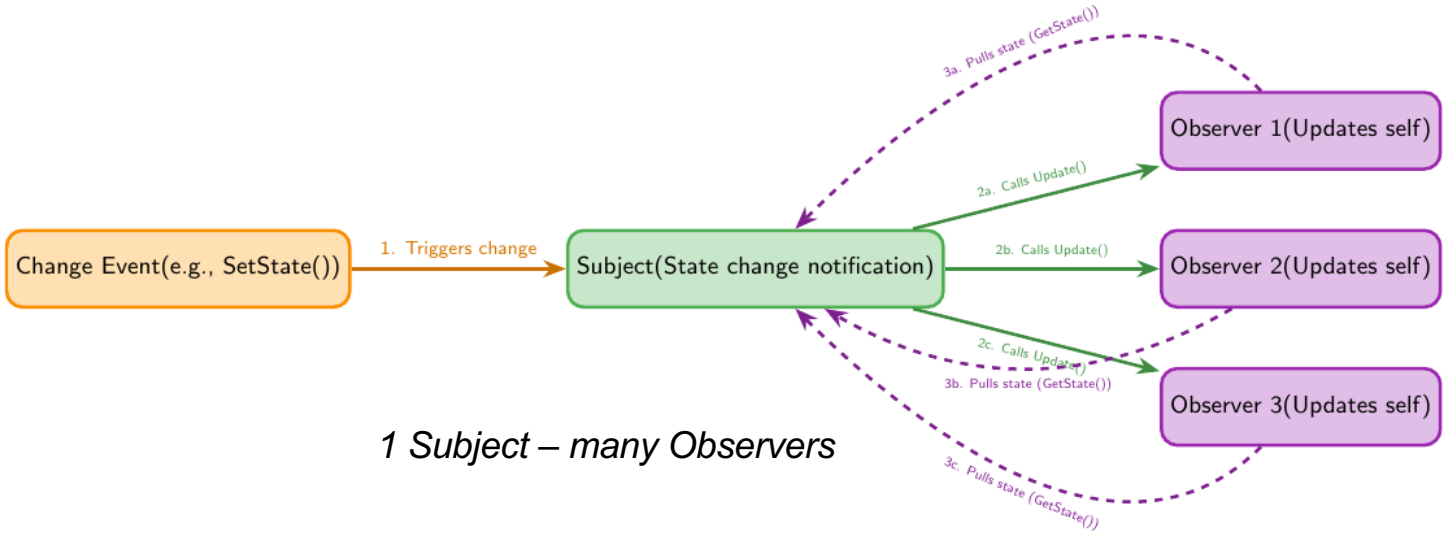


1 Subject – many Observers

The **Subject** notifies that something changed, and the **Observers** retrieve the specific data they need (through a getter). **(Pull Model)**



1 Subject – 1 Observer

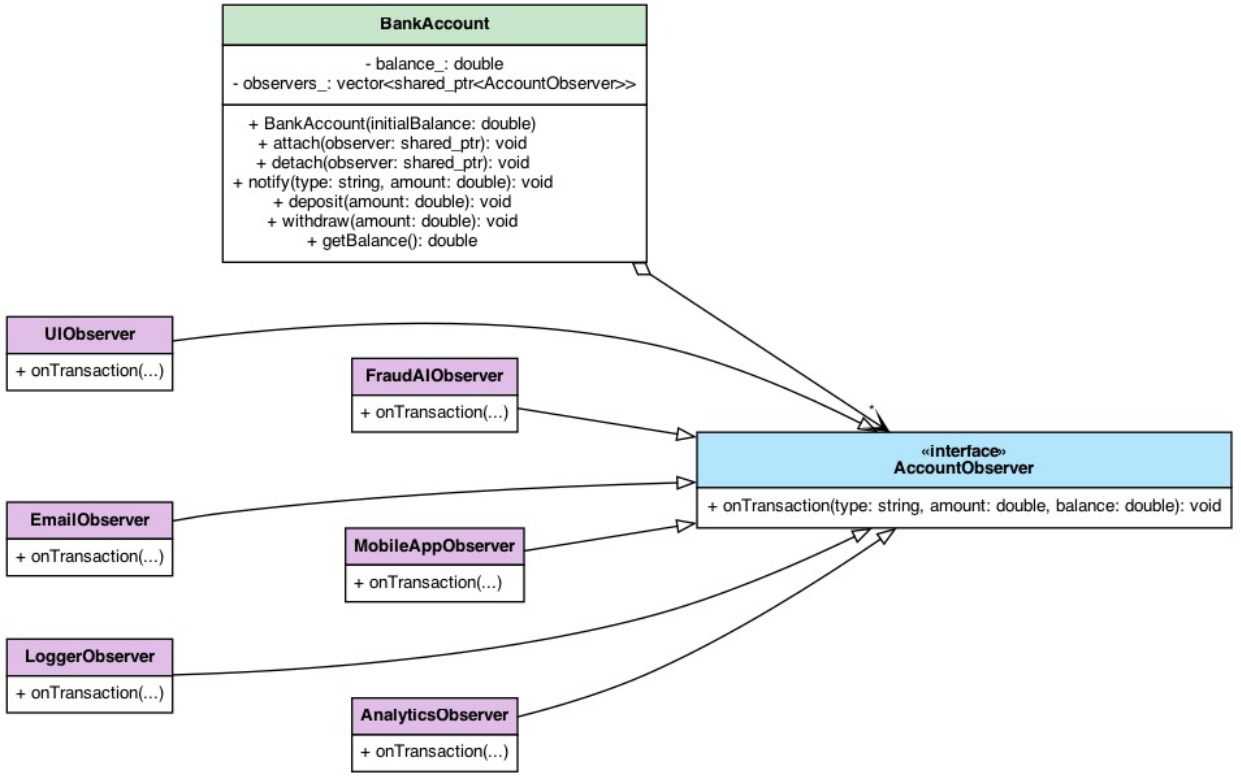


1 Subject – many Observers

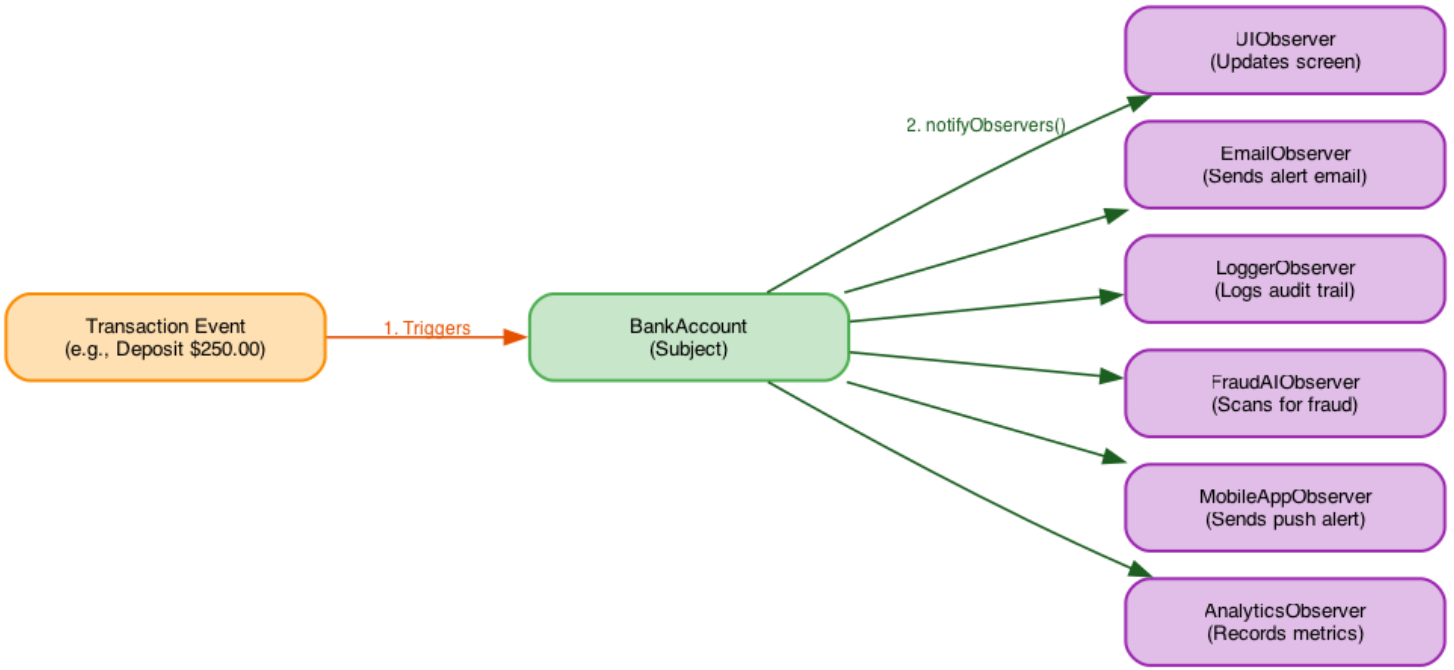
The **Subject** notifies that something changed, and the **Observers** retrieve the specific data they need (through a getter). **(Pull Model)**

Alternative: The Subject sends the changed data directly.
(Push Model)

Bank Account Example



Bank Account UML Diagram



- **True Isolation:** *BankAccount* is completely ignorant of the concrete services. It only knows they implement a generic observer interface.
- **Open/Closed Principle:** New services can be added without modifying a *BankAccount* class.
- **Single Responsibility:** Core financial logic remains clean. All secondary reactions (logging, fraud checks, UI) are delegated to dedicated components.
- **Runtime Flexibility:** Observers can be dynamically attached or detached on the fly based on user settings or system state.

Implementation & Demo

Naïve Approach Implementation

naive.hpp - Naive Bank Account Implementation

```cpp
// Naive Bank Account Implementation (Observer/src/naive.cpp)

#include <iostream>
#include <string>

// Tightly-coupled Naive Bank Account
class NaiveBankAccount {
private:
    double balance;
    UI ui;
    EmailSystem email;
    Logger logger;
    FraudDetector fraudDetector;
    MobileApp mobileApp;
    Analytics analytics;

public:
    NaiveBankAccount(double initialBalance = 0.0)
        : balance(initialBalance) {}

    void deposit(double amount) {
        balance += amount;

        // Direct, coupled method invocations on concrete systems
        ui.update(balance);
        email.sendEmail(amount);
        logger.log(amount, balance);
        fraudDetector.scan(amount);
        mobileApp.pushNotification(amount);
        analytics.trackActivity("Deposit", amount);
    }
};
```

Naïve Approach Demo

Terminal - bank_naive_demo

```
========================================
    BANK ACCOUNT DEMO: NAIVE VERSION
========================================
Step 1: Initializing NaiveBankAccount with $1000.00...

Step 2: Depositing $250.00. Expecting notifications to ALL 6 systems...

[UI] Balance display updated to $1250.00
[Email] Alert sent: A deposit of $250.00 was made.
[Logger] Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI] Scan complete: Deposit of $250.00 is marked OK.
[Mobile App] Push: Account credited with $250.00
[Analytics] Metric recorded: Event [Deposit] for $250.00

Step 3: Attempting to detach Email and Mobile App...
Error: Cannot detach components. NaiveBankAccount is tightly coupled
       to concrete notification systems.
========================================
```

Observer Design Pattern Implementation

pattern.hpp - Observer Pattern Interface Header

```cpp
// Core Observer Pattern Header Definition (Observer/src/pattern.cpp)

#include <iostream>
#include <memory>
#include <vector>
#include <string>

// Abstract Observer Interface
class AccountObserver {
public:
    virtual ~AccountObserver() = default;
    virtual void onTransaction(const std::string &type,
                               double amount,
                               double currentBalance) = 0;
};

// Subject Class Interface
class BankAccount {
public:
    BankAccount(double initialBalance = 0.0);

    void attach(std::shared_ptr<AccountObserver> observer);
    void detach(std::shared_ptr<AccountObserver> observer);
    void notify(const std::string &type, double amount);

    void deposit(double amount);
    void withdraw(double amount);
    double getBalance() const;

private:
    double balance_;
    std::vector<std::shared_ptr<AccountObserver>> observers_;
};
```

Observer Design Pattern Implementation

observers.hpp - Concrete Observers Definitions

```cpp
// Concrete Observers Implementation (Observer/src/pattern.cpp)

class UIObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                       double currentBalance) override {
        std::cout << "[UI Observer] Balance updated to $" << currentBalance << "\n";
    }
};

class EmailObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                       double currentBalance) override {
        std::cout << "[Email Observer] Alert sent for " << type << " of $" << amount << "\n";
    }
};

class LoggerObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                       double currentBalance) override {
        std::cout << "[Logger Observer] Logged: " << type << " of $" << amount << "\n";
    }
};

class FraudAIObserver : public AccountObserver {
public:
    void onTransaction(const std::string &type, double amount,
                       double currentBalance) override {
        if (amount > 10000.0)
            std::cout << "[Fraud AI] WARNING: Suspicious activity for $" << amount << "\n";
    }
};

class MobileAppObserver : public AccountObserver { /* Push notification delivery */ };
class AnalyticsObserver : public AccountObserver { /* Activity metrics recording */ };
```

Observer Design Pattern Demo Part 1

bank_observer_pattern_demo (Part 1)

```
========================================
    BANK ACCOUNT DEMO: PATTERN VERSION
========================================
=== RUNNING PATTERN DEMO (With Observer Pattern) ===
Step 1: Initializing BankAccount with $1000.00...

Step 2: Attaching observers...
 - UIObserver (updates screen balance display)
 - EmailObserver (sends transaction notification emails)
 - LoggerObserver (writes to security/audit logs)
 - FraudAIObserver (scans transaction amounts, warns if > $10,000)
 - MobileAppObserver (sends push notification)
 - AnalyticsObserver (records activity metrics)

Step 3: Depositing $250.00. Expecting notifications to ALL 6 systems...

[BankAccount]  Deposited $250.00. New balance: $1250.00
[UI Observer]  Balance display updated to $1250.00
[Email Observer]  Alert sent: A deposit of $250.00 was made.
[Logger Observer]  Transaction logged: Deposit of $250.00, new balance: $1250.00
[Fraud AI]  Scan complete:  Deposit of $250.00 is marked OK.
[Mobile App]  Push: Account credited with $250.00
[Analytics]  Metric recorded: Event [Deposit] for $250.00

Step 4: Dynamically detaching Email and Mobile App observers...
 (Customer disabled Email alerts and Mobile pushes in settings)
========================================
```

Observer Design Pattern Demo Part 2

bank_observer_pattern_demo (Part 2)

```
========================================
    BANK ACCOUNT DEMO: PATTERN VERSION
========================================
Step 5: Depositing $12000.00. Expecting notifications to remaining systems...
 (No Email, No Mobile push. Fraud AI alert should trigger for large amount)

[BankAccount]  Deposited $12000.00. New balance: $13250.00
[UI Observer]  Balance display updated to $13250.00
[Logger Observer]  Transaction logged: Deposit of $12000.00, new balance: $13250.00
[Fraud AI]  WARNING:  Suspicious large activity detected: Deposit of $12000.00
[Analytics]  Metric recorded: Event [Deposit] for $12000.00

Step 6: Customer re-enables Mobile App notifications in settings...
 (Attaching MobileAppObserver back to BankAccount)

Step 7: Withdrawing $500.00. Expecting notifications to UI, Logger, Fraud AI...

[BankAccount]  Withdrew $500.00. New balance: $12750.00
[UI Observer]  Balance display updated to $12750.00
[Logger Observer]  Transaction logged: Withdrawal of $500.00, new balance: $12750.00
[Fraud AI]  Scan complete:  Withdrawal of $500.00 is marked OK.
[Mobile App]  Push: Account debited with $500.00
[Analytics]  Metric recorded: Event [Withdrawal] for $500.00

Step 8: Attempting to withdraw $15000.00...
 (Expecting Insufficient Funds failure alerts to remaining observers)

[BankAccount]  Rejected: Insufficient funds for withdrawal of $15000.00
[UI Observer]  Balance display updated to $12750.00
[Logger Observer]  Transaction logged: FailedWithdrawal of $15000.00, new balance: $12750.00
[Fraud AI]  WARNING:  Suspicious large activity detected: FailedWithdrawal of $15000.00
[Mobile App]  Alert: Withdrawal of $15000.00 failed (Insufficient Funds).
[Analytics]  Metric recorded: Event [FailedWithdrawal] for $15000.00

=== PATTERN DEMO COMPLETED ===
========================================
```

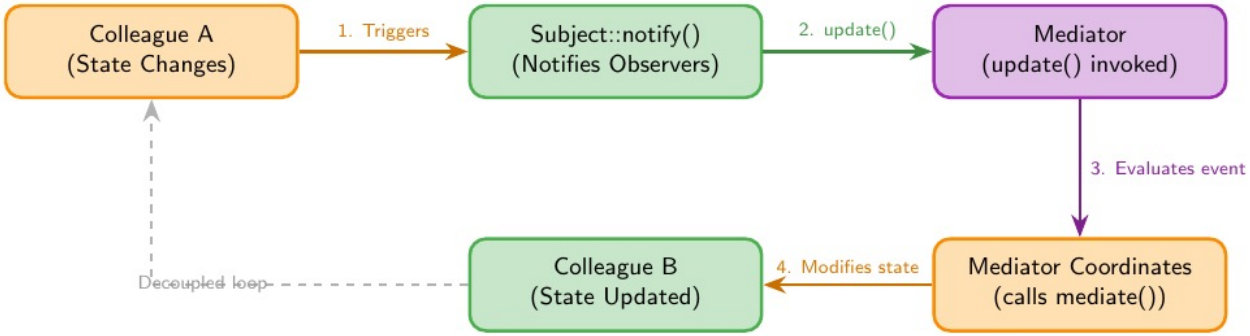
Relationship with Mediator Pattern

What is the Mediator Pattern?

- A behavioural design pattern that reduces chaotic dependencies between objects.
- It restricts direct communications between objects (Colleagues) and forces them to collaborate exclusively via a centralized mediator object.

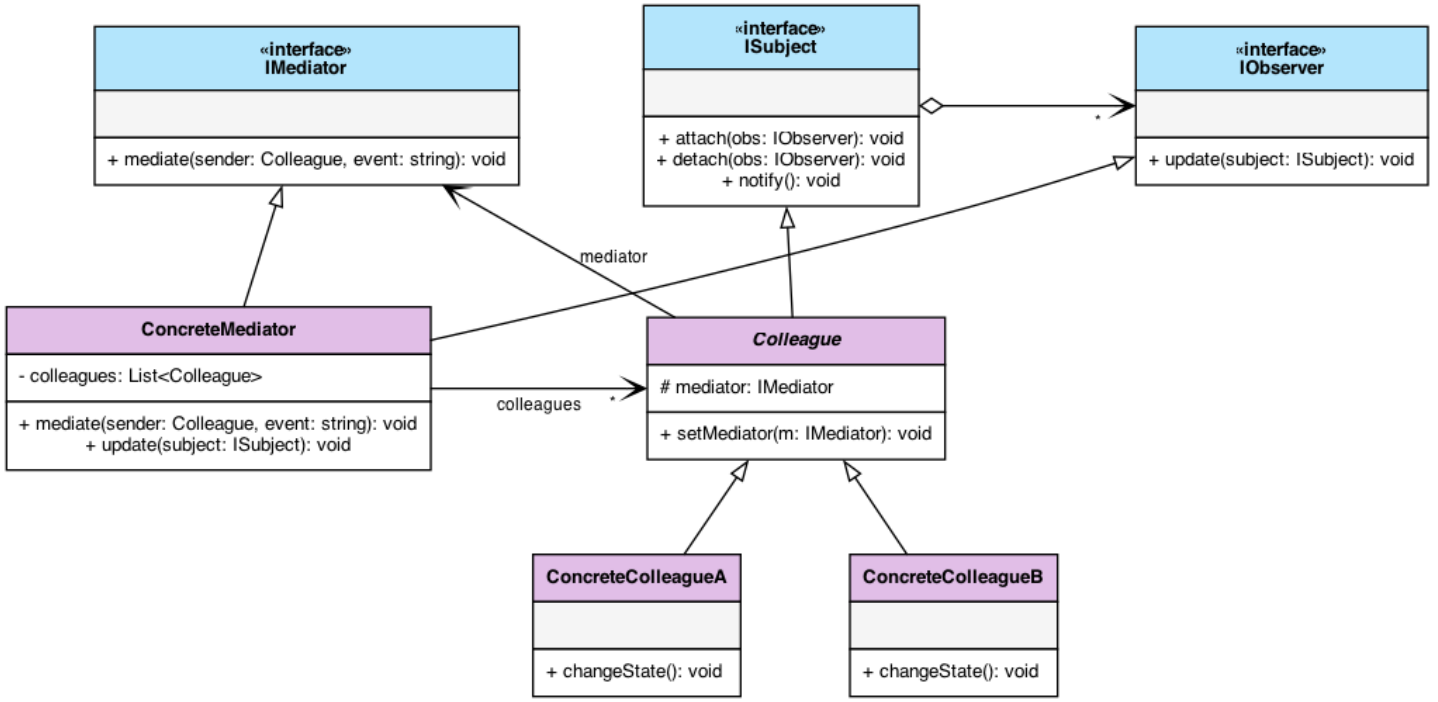
Airplanes communicate solely with the control tower (Mediator), which safely orchestrates the traffic.

Observer-Mediator Fusion



The Colleagues act as Subjects,
The Mediator acts as the Observer.

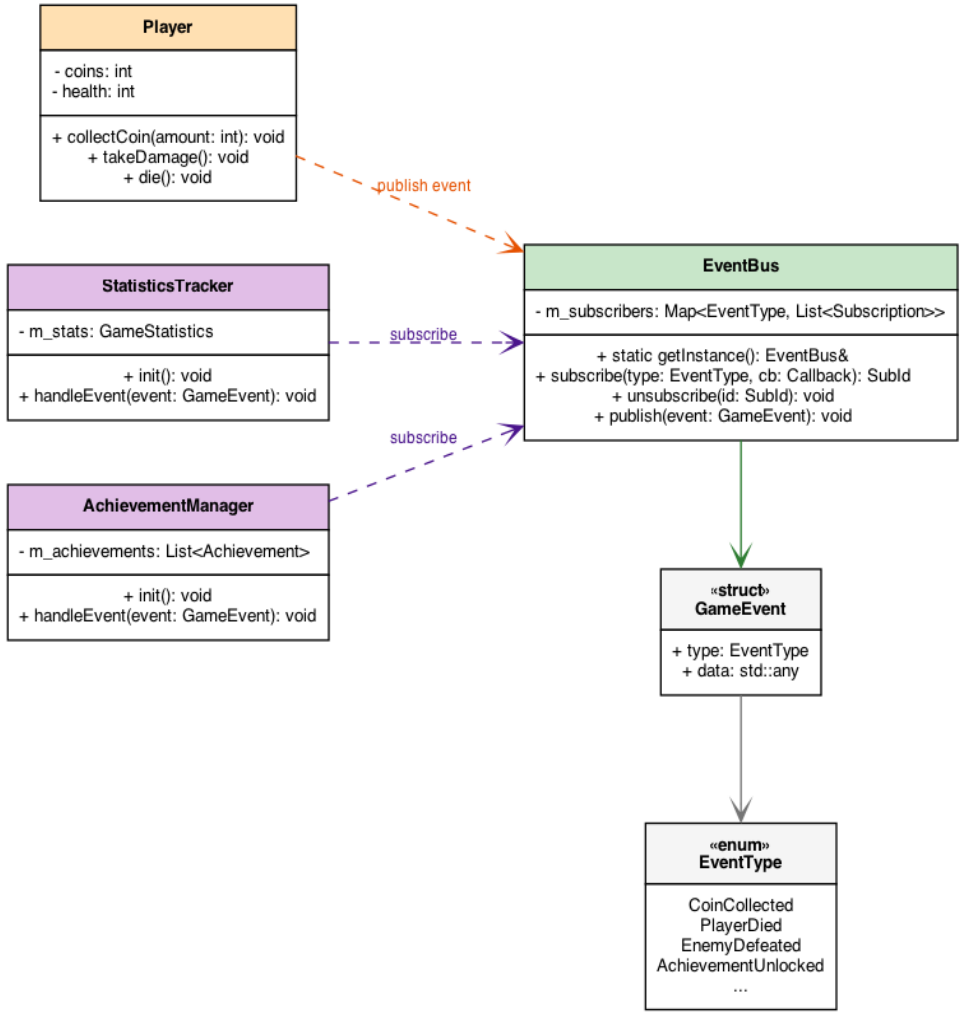
When a Colleague's state changes, it triggers an event. The Mediator (who is subscribed to these events) catches the notification, evaluates the global context, and updates other relevant Colleagues accordingly.



Observer-Mediator Fusion UML Diagram

EventBus in Game Development

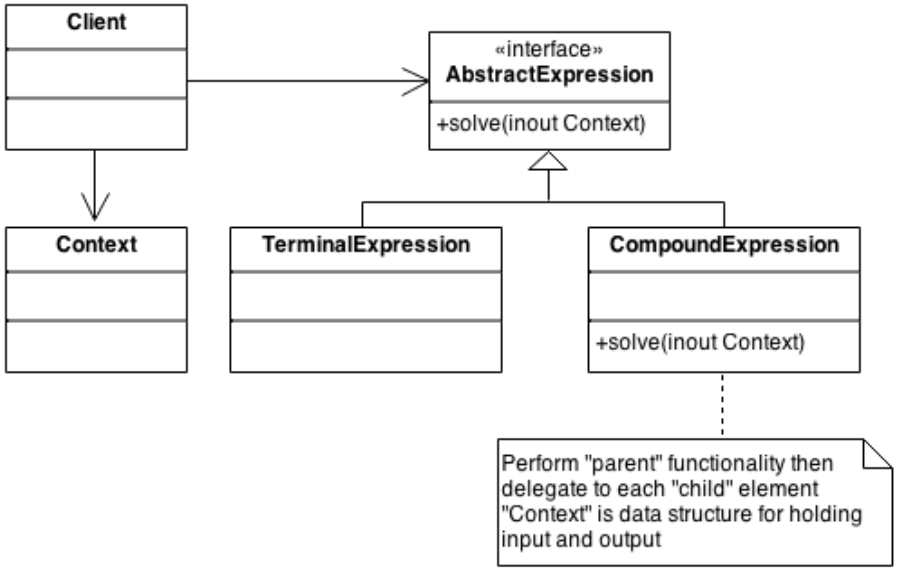
- A Player character takes damage and dies (Event Publisher).
- The Player does not call the AchievementManager or StatisticsTracker. It simply publishes a PlayerDied event to the central EventBus.
- The systems that subscribed to that specific event type are immediately notified and execute their logic.



Relationship with Interpreter design pattern

Interpreter design pattern: Given a language, we need to defined a hierarchical representation for the language.

Example: Representing (5 + 1) x 10 + 41 - x as an operation tree.



Source: https://sourcemaking.com/

Usage with Interpreter

However, if the terminal expression is a variable, which can be changed by another process, services, … instead of being a constant, then we need to update the result of the operations according to the new value.

=> We use observer pattern to observe the changes in the variable and update the tree accordingly.

Usage with Interpreter

Example: We have a spreadsheet (like an Excel file). In cell A3, we have a formula “= A1 + A2”. The result of A3 depends on the result of A1 and A2, which can be changed by another formula(s) or by user.

=> We use the interpreter pattern to interpret the formula in A3, and the observer pattern to observe the changes in A1 and A2.

Pros & Cons

Pros:

- **Open/Closed Principle:** New observer classes can be introduced without modifying the subject's code.
- **Loose Coupling:** The subject only interacts with a generic interface, remaining completely ignorant of the concrete observer implementations.
- **Dynamic Relationships:** Connections between publishers and subscribers can be established or terminated at runtime.

Pros & Cons

Cons:

- **The Lapsed Listener Problem:** If an observer is not explicitly detached, the subject retains a reference to it. This leads to dangling pointers and fatal crashes when the subject notifies deleted memory.
- **Non-Deterministic Execution:** The order in which subscribers are notified is usually arbitrary and cannot be relied upon for sequential logic.
- **Complex Debugging:** Tracing the exact flow of execution through a massive chain of dynamic, event-driven updates can be difficult to step through in a debugger.

Quiz

Quiz

Q1: If the Observer pattern were a real-world subscription service, which of the following scenarios best describes it?

A. A client polling a server every 5 seconds to check if there are new emails.

B. A newspaper publisher automatically delivering the daily issue to all registered home addresses.

C. A customer walking into a physical store every day to check if a shoe is back in stock.

D. A mediator forwarding calls between two people who don't know each other.

Quiz

Q1: If the Observer pattern were a real-world subscription service, which of the following scenarios best describes it?

A. A client polling a server every 5 seconds to check if there are new emails.

B. A newspaper publisher automatically delivering the daily issue to all registered home addresses.

C. A customer walking into a physical store every day to check if a shoe is back in stock.

D. A mediator forwarding calls between two people who don't know each other.

Quiz

Q2: In our naive Bank Account implementation, why did we violate the Open-Closed Principle (OCP)?

A. Because the BankAccount class was too small to hold the logic.

B. Because adding a new service required modifying the existing code of BankAccount.

C. Because we used C++ smart pointers instead of raw pointers.

D. Because the deposit() function was private.

Quiz

Q2: In our naive Bank Account implementation, why did we violate the Open-Closed Principle (OCP)?

A. Because the BankAccount class was too small to hold the logic.

B. Because adding a new service required modifying the existing code of BankAccount.

C. Because we used C++ smart pointers instead of raw pointers.

D. Because the deposit() function was private.

Quiz

Q3: Imagine a temporary UI window (observer) is deleted in C++ but forgets to call detach() or unsubscribe() on the BankAccount. What is the likely runtime result of the next deposit?

A. The compiler will fail with a syntax error.
B. The program will run faster because there's one less service.
C. The program will crash or experience undefined behavior (dangling pointer).
D. The bank account will automatically recreate the UI window.

Quiz

Q3: Imagine a temporary UI window (observer) is deleted in C++ but forgets to call detach() or unsubscribe() on the BankAccount. What is the likely runtime result of the next deposit?

A. The compiler will fail with a syntax error.

B. The program will run faster because there's one less service.

C. The program will crash or experience undefined behavior (dangling pointer).

D. The bank account will automatically recreate the UI window.

Quiz

Q4: If you are building a system where observers must be notified in a **strict, specific order** (e.g., Security Logger must run *before* UI Updates), is the standard Observer Pattern a good choice?

A. Yes, because the standard Observer pattern guarantees alphabetical notification order.

B. Yes, because observers are always notified in the exact order they subscribed.

C. No, because the Observer pattern is only meant for web browsers.

D. No, because the standard Observer pattern makes no assumptions about notification order, and enforcing it violates loose coupling.

Quiz

Q4: If you are building a system where observers must be notified in a **strict, specific order** (e.g., Security Logger must run *before* UI Updates), is the standard Observer Pattern a good choice?

A. Yes, because the standard Observer pattern guarantees alphabetical notification order.

B. Yes, because observers are always notified in the exact order they subscribed.

C. No, because the Observer pattern is only meant for web browsers.

D. No, because the standard Observer pattern makes no assumptions about notification order, and enforcing it violates loose coupling.

Quiz

Q5: In modern C++, if a Subject holds strong std::shared_ptr<Observer> references to its observers, and observers hold std::shared_ptr<Subject> back to the subject, what issue arises?

A. The compiler throws a syntax error.

B. A circular reference occurs, preventing objects from ever being destroyed (memory leak).

C. The program automatically converts them into raw pointers.

D. Observers are executed in reverse order.

Quiz

Q5: In modern C++, if a Subject holds strong std::shared_ptr<Observer> references to its observers, and observers hold std::shared_ptr<Subject> back to the subject, what issue arises?

A. The compiler throws a syntax error.

B. A circular reference occurs, preventing objects from ever being destroyed (memory leak).

C. The program automatically converts them into raw pointers.

D. Observers are executed in reverse order.

Quiz

Q6: What dangerous side effect occurs if an Observer's update() method mutates the Subject's state and triggers another deposit() without guard conditions?

A. Infinite recursive notification loop leading to a stack overflow crash.
B. The operating system increases process priority.
C. The balance doubles on each iteration automatically.
D. The observer is automatically removed from the subscriber list.

Quiz

Q6: What dangerous side effect occurs if an Observer's update() method mutates the Subject's state and triggers another deposit() without guard conditions?

A. Infinite recursive notification loop leading to a stack overflow crash.
B. The operating system increases process priority.
C. The balance doubles on each iteration automatically.
D. The observer is automatically removed from the subscriber list.

Quiz

Q7: How can the Interpreter pattern and Observer pattern work together in an automated financial trading system?

A. The Interpreter parses custom user trading rules (e.g. price > 100 AND volume > 500), and the Observer pattern broadcasts stock price updates to trigger the rule evaluation.

B. They cannot be used in the same project.

C. Observer compiles C++ code, while Interpreter runs the executable.

D. Interpreter deletes the Observer's memory pointers.

Quiz

Q7: How can the Interpreter pattern and Observer pattern work together in an automated financial trading system?

A. The Interpreter parses custom user trading rules (e.g. price > 100 AND volume > 500), and the Observer pattern broadcasts stock price updates to trigger the rule evaluation.

B. They cannot be used in the same project.

C. Observer compiles C++ code, while Interpreter runs the executable.

D. Interpreter deletes the Observer's memory pointers.

Quiz

Q8: What happens if a concrete observer calls subject->detach(this) inside its own update() callback while notify() is running a simple range-for loop?

A. The loop safely skips the detached observer.
B. The C++ compiler creates a vector copy automatically.
C. The program prompts the user for manual confirmation.
D. Iterator invalidation / undefined behavior because modifying a std::vector during range-based iteration corrupts the loop.

Quiz

Q8: What happens if a concrete observer calls subject->detach(this) inside its own update() callback while notify() is running a simple range-for loop?

A. The loop safely skips the detached observer.
B. The C++ compiler creates a vector copy automatically.
C. The program prompts the user for manual confirmation.
D. Iterator invalidation / undefined behavior because modifying a std::vector during range-based iteration corrupts the loop.

Real world problem

Moodle notification system

The Learning management system (LMS) Moodle need a notification system to notify the announcements and updates to the student.

Your task is to design the notification system for Moodle, which support a wide variety of notification, such as announcements, new classes, new tasks and/or quiz.

References

E. Gamma, R. Helm, R. Johnson, and J. Vlissides, *Design Patterns: Elements of Reusable Object-Oriented Software*. Reading, MA, USA: Addison-Wesley, 1994.

A. Shvets, "Observer Design Pattern," *Refactoring.Guru*. [Online]. Available: https://refactoring.guru/design-patterns/observer.

Thanks for watching